

Lab 1 Socket Programming — Stateful Client-Server Chat Application

Recording demonstrating

Connection

Server:

```
PS D:\FSCT\FSCT-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1> python .\server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 46264)
```

Client:

```
PS D:\FSCT\FSCT-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1> python .\client.py
Enter your username: █
```

The client successfully established a TCP connection with the server, and the server logged the incoming connection address.

Multiple messages

Server:

```
PS D:\FSCT\FSCT-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1> python .\server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 46264)
Received: HELLO|Kuangda
Username: Kuangda
Received: MSG|Hello World.
Kuangda says: Hello World.
Received: MSG|This is the second msg
Kuangda says: This is the second msg
Received: MSG|Trying some multiple messages.
Kuangda says: Trying some multiple messages.
Received: MSG|Thank you.
Kuangda says: Thank you.
█
```

Client:

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab1> python .\client.py
Enter your username: Kuangda
Server: OK|Hello Kuangda
Enter message or type EXIT to leave: Hello World.
Server: OK|Message received from Kuangda
Enter message or type EXIT to leave: This is the second msg
Server: OK|Message received from Kuangda
Enter message or type EXIT to leave: Trying some multiple messages.
Server: OK|Message received from Kuangda
Enter message or type EXIT to leave: Thank you.
Server: OK|Message received from Kuangda
Enter message or type EXIT to leave: 
```

The client successfully logs in and sends multiple messages with the server, with each message acknowledged correctly.

Protocol behavior

As shown in the last paragraph, the protocol behavior is correctly demonstrated: the server successfully handles the client's login handshake (HELLO) to establish the user state and properly processes subsequent chat messages (MSG).

Robustness tests

Empty message

Server:

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab1> python .\server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 51444)
Received: HELLO|Kuangda
Username: Kuangda
Received: MSG|
```

Client:

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab1> python .\client.py
Enter your username: Kuangda
Server: OK|Hello Kuangda
Enter message or type EXIT to leave:
Server: ERROR|Message cannot be empty
Enter message or type EXIT to leave: 
```

As shown in the screenshot, the server successfully detects and handles an empty

Message longer than 200 characters

[illegible][illegible]

Unknown command

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab1> python .\server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 52819)
Received: HELLO|Kuangda
Username: Kuangda
Received: BADCOMMAND|test
```

Client:

```
Enter your username: Kuangda
Server: OK|Hello Kuangda
Enter message or type EXIT to leave: test
Server: ERROR|Unknown command
Enter message or type EXIT to leave: 
```

As shown in the screenshot, the server successfully detects and rejects the unknown command with an error response, while keeping the client running.

MSG sent before HELLO

```
# Create a command to test Robustness(Skip the login)
if username.upper() == "SKIP":
    print("Skipping HELLO command...")
_
```

I modified this part to allow skipping the login step for robustness testing.

Server:

```
PS D:\FSCT\FSC8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1> python .\server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 41583)
Received: MSG|Hello, I did not log in
_
```

Client:

```
PS D:\FSCT\FSC8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1> python .\client.py
Enter your username: skip
Skipping HELLO command...
Enter message or type EXIT to leave: Hello, I did not log in
Server: ERROR|HELLO required first
Enter message or type EXIT to leave: 
```

As shown in the screenshot, the server successfully detects and rejects the message with an error response when the client tries to send a message without logging in, while keeping the client running

Clean disconnection

Server:

```
Received: EXIT|
Server closed
PS D:\FSCT\FSC8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1> 
```

Client:

```
Enter message or type EXIT to leave: exit
Server: OK|Goodbye
Disconnected
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1> 
```

As shown in the screenshot, the client sends an EXIT command to disconnect cleanly. The server successfully receives it, responds with OK|Goodbye, and closes the connection properly.

Security analysis

In this lab, the server implements strict security and robustness mechanisms against malicious or malformed inputs. The server checks for the delimiter | and rejects unrecognized commands (e.g., BADCOMMAND) with an error response, preventing crashes. The server also enforces session state checks, requiring a HELLO login before accepting any chat messages (MSG). Unauthorized messages are blocked. Overall, the system demonstrates strong robustness and resilience against multiple types of attacks.

Reflection answers

1. What happens if the server crashes while the client is connected?

The client returns an error and disconnects.

2. How could the server be extended to handle multiple clients?

By using threading.

3. Why can `recv(1024)` split messages unexpectedly?

Because messages can exceed the buffer size, causing them to be split into chunks.

4. Why does TCP reliability not mean the application is secure?

Because TCP only ensures data delivery, not application-level security or logic.

5. What new attack surfaces are created by adding usernames and protocol commands?

Malformed inputs, login bypass, and command injections.

6. How could a malicious client abuse the protocol?

By sending injection payloads or flooding the server with multiple requests (DoS).

7. How would you add authentication?

By storing user credentials in a database and requiring users to log in.

8. How would you add encryption?

By using TLS/SSL to encrypt the network traffic.

9. What validation is performed in your implementation?

Checking for the | delimiter, enforcing HELLO login first, and validating input length.

10. What additional mitigations would be needed for a real deployment?

Require a password during login and check if the username is already taken to prevent duplicates.

Challenge

Requirements 1&2:

Server:



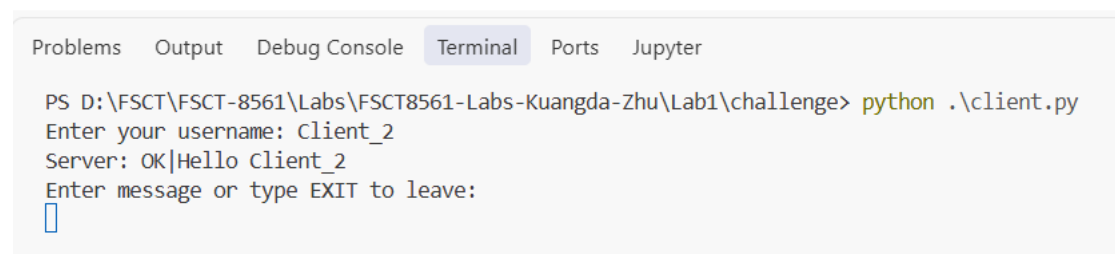
```
Problems Output Debug Console Terminal Ports Jupyter
PS D:\FSCT\FSCT-8561\Labs\FST8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\server.py
Server is waiting for a connections...
Connected by: ('127.0.0.1', 17986)
Received: HELLO|Client_1
Username: Client_1
Connected by: ('127.0.0.1', 17988)
Received: HELLO|Client_2
Username: Client_2
█
```

Client1:



```
Problems Output Debug Console Terminal Ports Jupyter
PS D:\FSCT\FSCT-8561\Labs\FST8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\client.py
Enter your username: Client_1
Server: OK|Hello Client_1
Enter message or type EXIT to leave:
█
```

Client2:



```
Problems Output Debug Console Terminal Ports Jupyter
PS D:\FSCT\FSCT-8561\Labs\FST8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\client.py
Enter your username: Client_2
Server: OK|Hello Client_2
Enter message or type EXIT to leave:
█
```

As we can see from the screenshot above, two clients are connected to the server at the same time. They provide their usernames using the format HELLO|Client_1 and HELLO|Client_2.

Requirements 3&4&5:

Server:

```
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\server.py
Server is waiting for a connections...
Connected by: ('127.0.0.1', 17986)
Received: HELLO|Client_1
Username: Client_1
Connected by: ('127.0.0.1', 17988)
Received: HELLO|Client_2
Username: Client_2
Received: MSG|Hello, I am Client 1
Client_1 says: Hello, I am Client 1
Received: MSG|Nice to meet you, I am Client 2
Client_2 says: Nice to meet you, I am Client 2
█
```

Client1:

```
Problems Output Debug Console Terminal Ports Jupyter
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\client.py
Enter your username: Client_1
Server: OK|Hello Client_1
Enter message or type EXIT to leave:
Hello, I am Client 1
Client_2: Nice to meet you, I am Client 2
█
```

Client2

```
Problems Output Debug Console Terminal Ports Jupyter
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\client.py
Enter your username: Client_2
Server: OK|Hello Client_2
Enter message or type EXIT to leave:
Client_1: Hello, I am Client 1
Nice to meet you, I am Client 2
█
```

From the screenshot above, we can see that both usernames (Client_1 and Client_2) are saved on the server. Also, the clients can send messages, and the server forwards the messages along with the sender's username to other connected clients.

Requirements 6:

Server:

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

Received: HELLO|Client_1
Username: Client_1
Connected by: ('127.0.0.1', 17988)
Received: HELLO|Client_2
Username: Client_2
Received: MSG|Hello, I am Client 1
Client_1 says: Hello, I am Client 1
Received: MSG|Nice to meet you, I am Client 2
Client_2 says: Nice to meet you, I am Client 2
Received: EXIT|
Connection reset by client
Connection closed for ('127.0.0.1', 17988)
Received: MSG|Good bye, Client 2
Client_1 says: Good bye, Client 2
█
```

Client1:

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\client.py
Enter your username: Client_1
Server: OK|Hello Client_1
Enter message or type EXIT to leave:
Hello, I am Client 1
Client_2: Nice to meet you, I am Client 2
Good bye, Client 2
█
```

Client2:

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\client.py
Enter your username: Client_2
Server: OK|Hello Client_2
Enter message or type EXIT to leave:
Client_1: Hello, I am Client 1
Nice to meet you, I am Client 2
Exit

Disconnected
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge> █
```

From the screenshot, we can see that after Client_2 disconnects, the server continues

running, and Client_1 can still send messages.

Requirements 7:

Server:

```
Connected by: ('127.0.0.1', 17988)
Received: HELLO|Client_2
Username: Client_2
Received: MSG|Hello, I am Client 1
Client_1 says: Hello, I am Client 1
Received: MSG|Nice to meet you, I am Client 2
Client_2 says: Nice to meet you, I am Client 2
Received: EXIT|
Connection reset by client
Connection closed for ('127.0.0.1', 17988)
Received: MSG|Good bye, Client 2
Client_1 says: Good bye, Client 2
Connection reset by client
Connection closed for ('127.0.0.1', 17986)
█
```

Client1:

```
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge> python .\client.py
Enter your username: Client_1
Server: OK|Hello Client_1
Enter message or type EXIT to leave:
Hello, I am Client 1
Client_2: Nice to meet you, I am Client 2
Good bye, Client 2
Traceback (most recent call last):
  File "D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge\client.py", line 48, in <module>
    message = input("")
KeyboardInterrupt
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab1\challenge> █
```

From the screenshot, we can see that when Client_1 unexpectedly disconnects, the server continues running well without crashing.

Challenge Reflection

1. Why could the original Lab 1 server handle only one client at a time?

Because it did not use multithreading, so it could only handle one connection at a time.

2. What problem does threading solve?

Threading allows a program to do multiple things simultaneously.

3. What happens to the other clients when one client disconnects?

The other clients can continue running and communicating.

4. What additional security risks arise when multiple clients share the same server?

It is vulnerable to DDoS attacks.

5. What could happen if two clients try to use the same username?

It could lead to identity spoofing.

6. How would you prevent one user from impersonating another user?

By prohibiting duplicate usernames.

Github

<https://github.com/bnwzc/FSCT8561-Labs-Kuangda-Zhu>